IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 8 August 2024, accepted 3 September 2024, date of publication 10 September 2024, date of current version 23 September 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3457587

RESEARCH ARTICLE

SS-DBSCAN: Semi-Supervised Density-Based Spatial Clustering of Applications With Noise for Meaningful Clustering in Diverse Density Data

TIBA ZAKI ABDULHAMEED[1], (Member, IEEE), SUHAD A. YOUSIF[1], VENUS W. SAMAWI[2], AND HASNAA IMAD AL-SHAIKHLI[1]

[1]Computer Science Department, College of Sciences, Al-Nahrain University, Baghdad 64074, Iraq
[2]Department of Smart Business, Isra University, Amman 11622, Jordan

Corresponding author: Suhad A. Yousif (suhad.a.yousif@nahrainuniv.edu.iq)

ABSTRACT DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised clustering algorithm designed to identify clusters of various shapes and sizes in noisy datasets by pinpointing core points. The primary challenges associated with the DBSCAN algorithm involve the recognition of meaningful clusters within varying densities datasets and its sensitivity to parameter values of Epsilon distance and minimum number of neighbor points. These two issues may result in merging small clusters into larger clusters or splitting valid clusters into smaller clusters. A new Semi-Supervised DBSCAN (SS-DBSCAN) algorithm is introduced to improve the recognition of meaningful clusters. DBSCAN requires core points to be within, at most, Epsilon distance from their minimum neighboring points. The SS-DBSCAN algorithm, a modified version of the original DBSCAN, adds a pre-specified condition or constraint to identify core points further. This extra constraint is related to the clustering objective of a given dataset. To evaluate the effectiveness of SS-DBSCAN, we utilize three datasets: letter recognition, wireless localization, and Modern Standard Arabic (MSA) combined with Iraqi words language modeling. V-measure is used to evaluate the clustering efficiency for the letters recognition and wireless localization datasets. The perplexity (pp) of the class-based language model, built on the produced clusters, is the metric used for the Iraqi-MSA dataset clustering effectiveness. Experimental results showed the significant effectiveness of SS-DBSCAN. It outperforms DBSCAN when applied to letters and Iraqi-MSA datasets with improvements of 65% and 14.5%, respectively. A comparable performance was achieved when clustering the wireless localization dataset. Additionally, to assess the effectiveness of SS-DBSCAN, its performance has been compared to various modified versions of DBSCAN using four metrics: V-measure, PP, Adjusted Rand Index (ARI), and the Silhouette score. Based on these metrics, the results showed that SS-DBSCAN outperformed most DBSCAN versions in three case studies. Consequently, the proposed SS-DBSCAN algorithm is particularly suitable for high-density datasets. The SS-DBSCAN python code is available at https://github.com/TibaZaki/SS_DBSCAN.

INDEX TERMS Clustering, DBSCAN, semi-supervised clustering, unsupervised classification, word classification.

I. INTRODUCTION

Clustering is a technique that splits a set of abstract objects into clusters, each containing objects with high intra-cluster similarity and low inter-cluster similarity. It is used to extract useful patterns from complex data sources. Clustering aims to combine associated objects (points) into clusters without prior knowledge [1]. The clustering analysis is crucial in machine learning and data mining techniques. It is utilized for data reduction, pattern recognition, data segmentation, web

The associate editor coordinating the review of this manuscript and approving it for publication was Claudia Raibulet.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

search, and outlier detection [2]. Different clustering methods are defined in the literature [3]. They are categorized as partitioning clustering [3], [4], [5], hierarchical clustering [6], density-based clustering [7], model-based clustering [8], grid-based clustering [9], and mixture model clustering [10], [11]. The main focus of this paper is the density-based clustering category.

In the density-based clustering technique, a cluster continues to grow as long as the density of points is greater than a threshold value, and the low-density outer points are considered noise points. The data points clustering is based on density reachability and connectivity. The reachability is established when two points reside within a certain distance of each other. Furthermore, connectivity is established when all points in the path (connecting these two points) are density reachable from the nearest points in the same path [12].

DBSCAN is one of the most widely used density-based methods for two features. First, the number of clusters does not need to be defined as an input parameter. Second, clusters with arbitrary density shapes can be discovered [13]. However, in some cases, the clusters' shapes can be very complicated, causing the incorporation of small clusters into a few larger clusters or splitting valid clusters into several small clusters.

This paper addresses the mentioned limitation by enhancing the DBSCAN algorithm's efficiency in generating clusters for dense data. It does this by prioritizing certain object features over others instead of considering all features equally important. Consequently, the main contribution of this work is the introduction of a Semi-Supervised clustering algorithm called SS-DBSCAN. SS-DBSCAN is classified as a semi-supervised algorithm as it requires initial information about the core point. It introduces an additional condition to specify a core point by incorporating the Is-important(point) function into the DBSCAN algorithm. By specifying certain conditions related to the clustering purpose, this algorithm resolves the problem of invalid cluster identification within a dataset, particularly when the dataset comprises clusters of varying densities.

To explain the behaviour of the SS-DBSCAN algorithm compared to the original DBSCAN algorithm, three types of dataset were tested: images, wireless signals, and text. These data show the performance of the proposed algorithm on noisy and noiseless data. The wireless localization benchmark dataset [14] is noisy. In contrast, the letters recognition dataset [15] and an actual data application, "Modern Standard Arabic (MSA) with Iraqi words" [16], are noiseless data. The clustering behavior of the letter recognition and wireless localization datasets was assessed using V-measure, Silhouette, and Adjusted Rand Index (ARI). The Iraqi-MSA dataset clustering efficiency is evaluated using the perplexity (pp) of the language model based on the produced clusters and the Silhouette metrics. It was noted that the V-measure and the ARI metrics are unsuitable for unclassified datasets, such as the Iraqi-MSA dataset.

The following sections are organized as follows. First, the related works are presented in section II. Then, the main concept of the proposed SS-DBSCAN algorithm is described in section III along with an illustrative example. Section IV outlines the steps followed to create and evaluate SS-DBSCAN (system model). Next, in section V, the experimental setup for three case studies is illustrated. Finally, results are presented and discussed in section VI, followed by the conclusion in section VII.

II. RELATED WORK

Much research was concerned with the enhancements applied to the DBSCAN algorithm to improve the underlying datasets' clustering analysis and algorithm complexity in terms of time and space [5], [17]. In 1996, [7] developed the DBSCAN density-based clustering algorithm to recognize clusters of arbitrary shape. As with most clustering algorithms, DBSCAN has problems with specifying (minimum number of neighbor points *MinPts* and values of Epsilon distance *Eps*), algorithm complexity, and variant density cluster within the dataset [3], [5], [7]. Since then, researchers have attempted to overcome the drawbacks of the DBSCAN algorithm and enhance its features. Some enhancements focused on avoiding the specification of the density thresholds (minimum number of neighbor points *MinPts*, and values of Epsilon distance *Eps*) by the users [1], [18], VDBSCAN [19], GRIDBSCAN [20], [21], SA-DBSCAN [22], IDBSCAN [23], and ST-DBSCAN [24], AEDBSCAN [25], and [26]. Researchers also handle the algorithm complexity problem concerning execution time and memory [23], [27], [28]. Parallel implementation of the DBSCAN algorithm is applied to large-size datasets [21], [29]. Other enhancements applied to DBSCAN handle the problem of limiting the amount of allowed local density variation [30], [31], and variant density clusters [19], [20], which we will concentrate on in this section.

A Hierarchical version of DBSCAN (H-DBSCAN) proposed by [32] is developed to handle noise, outliers, and clusters of varying sizes and shapes. Unlike DBSCAN, H-DBSCAN focuses on high-density clustering, which reduces noise clustering and enables hierarchical clustering based on a decision tree approach. H-DBSCAN clustering offers automatic cluster discovery, identification of clusters of different shapes and sizes, and a hierarchical clustering structure. However, it can be computationally intensive for large datasets, sensitive to distance metrics, and requires parameter tuning. Reverse nearest neighbours and k-reverse nearest neighbours are also used to improve the clustering process. Record [33] detects outlier and stable clusters that can be obtained using kreverse-nearest neighbors. RNN-DBSCAN [34] uses k-nearest and reverse nearest neighbors to expand the cluster. KR-DBSCAN [13] is a developed version of RNN-DBSCAN based on reverse nearest neighbour and influence space to improve discriminating the neighboring clusters of different densities. Influence space

is also used by IS-DBSCAN [35] to identify density and reachability. ISB-DBSCAN [36] is a developed version of IS-DBSCAN to cluster non-core objects when non-core objects are equidistant from two core objects.

Adding conditions manually in a semi-supervised DBSCAN is also proposed by [37], achieving clustering efficiency improvement through must-be-linked and cannot-be-linked conditions.

Authors in [24] developed an improved version of DBSCAN (called ST-DBSCAN algorithm) based on changing the distance computation and producing the cluster density factor and object value. Some noise points are not detected when the dataset contains clusters with different densities. Consequently, in the ST-DBSCAN algorithm, a density function is introduced for each cluster to detect noise points. ST-DBSCAN is proven effective when applied to a dataset with spatial-temporal data characteristics. A combination of the K-means clustering followed by DBSCAN was proposed by [38]. The K-means were applied first to produce the subclusters, and then these subclusters were merged to produce the end clusters. DBSCAN was applied to merge the subclusters using a proposed cluster distance matrix. Clustering time was reduced while keeping efficient clustering. An improved version of DBSCAN named dual grid-based DBSCAN was proposed by [39] that tackles the issue of choosing the best values for the parameters by the use of the inner grid and, in addition, reducing the computation time through the use of the outer grid of the dataset. Another Grid-based improvement was proposed by [40], where the size and shape of the grid allow adjusting the parameters for each area in the grid. Local clustering is performed before merging the final data. A novel adaptive density-based spatial clustering algorithm (Ada-DBSCAN) was developed to address the problem of linear connection that causes poor discrimination of clusters correlated to some data points. Ada-DBSCAN includes a data block splitter and a data block merger coordinated by local and global clustering. Both synthetic and real-world datasets are used to evaluate Ada-DBSCAN efficiency. The experimental result showed that Ada-DBSCAN outperforms the DBSCAN algorithm in clustering analysis accuracy [41]. An extended version of the DBSCAN algorithm was proposed by [42] that employs a dynamic radius and assigns each object a regional density value before counting objects with a similar density inside the radius. The object is a core, and a cluster can grow from it if the neighborhood size is greater than or equal to *MinPts*; otherwise, the object is temporarily assigned noise. Two items have similar local densities if their similarity exceeds the threshold. With the proposed approach, clusters of any density can be successfully found in the data. The experimental results demonstrate the superior ability of the proposed method to detect clusters of varying densities even when there are no noticeable gaps between them. A partitioning technique called IMSTAGRID is proposed by [8] to enhance the density-cube-based data model and improve its accuracy. IMSTAGRID is developed based on adapting the partitioning process, determining the distance threshold, and calculating the density compensation during clustering.

Accordingly, it is seen that researchers used distinct techniques to improve the DBSCAN algorithm in terms of time complexity, automatic specification of the density thresholds (*MinPts*, and *Eps*), and variant density clusters within a dataset (SS-DBSCAN algorithm main focus). The SS-DBSCAN algorithm solves the problem of invalid cluster identification within a dataset, especially when the dataset includes clusters of diverse densities. This is accomplished by modifying DBSCAN to be a semi-supervised clustering algorithm, in which some pre-specified conditions related to the clustering purpose of the data are added. Those pre-specified conditions shed light on points that are more important or more distinct than others. The user specifies their importance depending on the problem dataset and the goal behind the whole clustering process. For instance, a pre-specified distance condition from the mean could be applied to numerical data.

III. THE PROPOSED SS-DBSCAN: MAIN CONCEPT

To understand the proposed algorithm idea and steps, basic terms related to the DBSCAN algorithm need to be illustrated [24]:

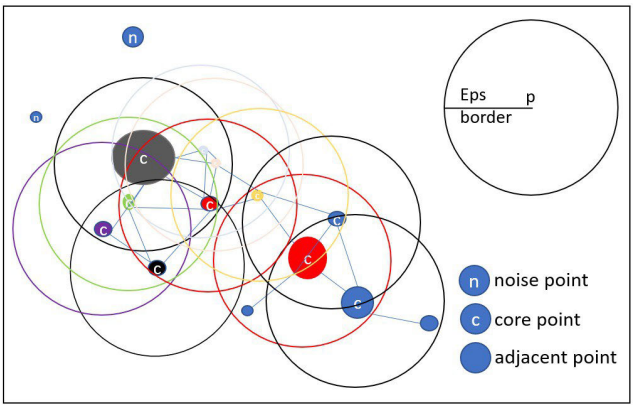
The **Core point** is a central point of a neighborhood that must include at least a minimum number of points (*MinPts*). Those points must not exceed an Epsilon distance (*Eps*) from their related core point. The original DBSCAN identifies a core point as shown in predicate logic (1):

$$\forall p\ \text{point}(p) \wedge \text{neighborCount}(p, n) \wedge (n \geq (\text{MinPts})) \implies \text{Is_core}(p) \tag{1}$$

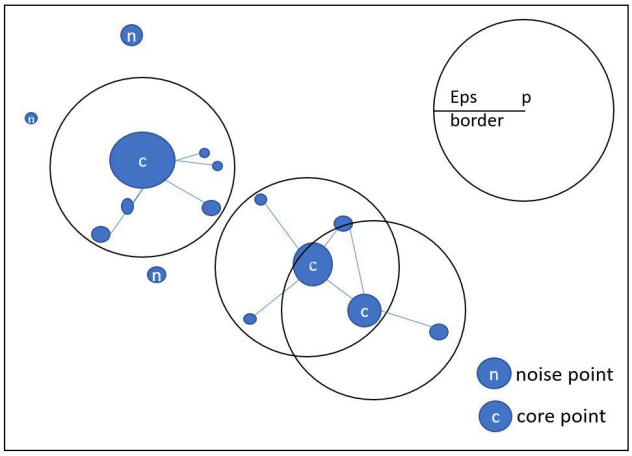
where p is any point, and $neighborCount(p, n)$ computes the number of points n, which represents neighbors of p and has a distance value that is less or equal to *Eps*.

The **Minimum Number of Points (*MinPts*)** is a pre-specified number of points that should occur in a neighborhood. **Epsilon distance (*Eps*)** is the maximum distance between a core point and its neighboring point. It also needs to be predefined. The **Neighbor point (adjacent point)** is determined by the Epsilon distance (*Eps*), such as the Manhattan distance or Euclidean distance for any two points p and q. In other words, the distance between any core point and its neighbor(s) must not exceed a predefined Epsilon distance (*Eps*). A Cluster is a dense of points connected directly or through one or more intermediate core points, where each cluster contains at least a minimum predefined point *MinPts*.

The SS-DBSCAN algorithm, presented in Algorithm (1), includes the additional condition for further specification of a core point underlined in line 17. It shows $Is_important(point)$ as a function that represents the extra condition, which is illustrated in Algorithm (2). The predicate logic (2) defines



(a) Marble balls clusters using DBSCAN algorithm



(b) Marble balls clusters using SS-DBSCAN algorithm

FIGURE 1. Marble balls clustering using DBSCAN and SS-DBSCAN algorithms.

the new specification of the core point condition:

$$\forall p\ \text{point}(p) \wedge (\text{neighborCount}(p, n) \wedge (n > (\text{MinPts})) \wedge \text{Is_important}(p)) \implies \text{Is_core}(p) \quad (2)$$

The importance of a point is decided according to the purpose of the clustering operation. Adding the condition(s) turns the DBSCAN into an SS-DBSCAN. This development allows the use of DBSCAN with higher attention to how the data is spread to form masses of various densities. This article presents three clustering case studies to demonstrate how to apply the "Is_ Important" condition in each clustering case. A detailed explanation of the three case studies is given in section V. The specification of the *Is_important* condition can be done by utilizing the following steps:

- **Step 1:** Specify clustering purpose or objective.
- **Step 2:** Define a criterion by which the clustering can be performed.
- **Step 3:** Identify the feature(s) that could be used to fulfill the criteria for clustering. If you could not directly identify the feature(s) that could achieve the criteria, then apply one of the feature selection methods to choose the feature(s).
- **Step 4:** Formulate the *Is_important* condition(s) using the key feature(s) to achieve clustering based on the specified criterion.

ALGORITHM 1 SS-DBSCAN

Require: Data D, Epsilon distance Eps, and Minimum number of points per cluster $MinPt$

```
DistM = Compute Distance Matrix(D)
Cluster = 0 #Set current cluster number to Zero
for point ∈ D do
    labels[point] = 0;
end for
for point ∈ D do
    if labels[point] == 0 then
        Neighboring_Points = FindAllNeighbores(point, Eps, DisM);
        if size of Neighboring_Points > MinPts then
            C = C+1# Grow a new cluster
            #Neighboring_Points is used as a FIFO queue, and i is its pointer
            i = 0;
            while i < size of Neighboring_Points do
                point = Neighboring_Points[i];
                if labels[point] == −1 then
                    labels[point] = C;
                else if labels[point] == 0 then
                    labels[point] = C # Find all the neighbours of the point.
                    NewNeighbors = FindAllNeighbores(point, Eps, DisM);
                    Is_core = size of (NewNeighborPts) >= MinPts AND Is_important(point);
                    if Is_core == true then
                        # Expand the neighbouring list
                        Neighboring_Points = Neighboring_Points + NewNeighbors;
                    end if
                end if
                i = i + 1 # Advance the Neighboring_Points queue pointer to the next point
            end while
        else
            labels[point] = −1 # define this point as noise
        end if
    end if
end for
return labels
```

These steps are detailed for the three case studies tested in this article.

For a better understanding of the SS-DBSCAN algorithm behaviour, let us assume we have 15 marble balls of different sizes and locations that need to be clustered.

ALGORITHM 2 Is-Important

Require: point
1: **if** *PointCondition* == *true* **then**
    # The condition is specified by the user
    # such as predicate (3)
2: **return** *True*
3: **else**
4: **return** *False*
5: **end if**

Each marble ball (p) is defined by two features: the size, defined by the Marble radius (*Mradius*), and its location, which is defined by an (x, y) coordinate.

The *Is_important* condition is set using the listed steps mentioned above. Thus, these are the steps to decide the marbles additional *Is_important* condition.

- Step 1: The clustering purpose is to group the marbles based on their locations, having clusters formulated around big marbles only.
- Step 2: The criteria for determining the importance of a marble is based on its size.
- Step 3: The ball radius feature is selected. A marble is considered important if its radius is greater than twice the average radius of all the marbles. In other words, the minimum radius for a marble to be considered important is twice the average radius of all the marbles.
- Step 4: Then, the formulated *Is_important* condition is as illustrated in predicate (3):

$$\forall p\ \text{marble}(p) \wedge \text{Mradius}(p, r) \wedge \$r > (2 \times \text{meanMradius}))) \implies \text{Is_important}(p) \quad (3)$$

where each marble ball (point p) has an *Mradius*, and p can be considered as a core marble ball if *Mradius* is big enough. The noise is decided according to its distance from the points included in the cluster. Thus, clusters do not include noise points.

Fig. 1 shows marble balls clustering using DBSCAN and SS-DBSCAN algorithms. In DBSCAN, clusters may spread beyond the desired area, as shown in Fig. 1a, where points are clustered into one big cluster with 11 core points out of 15. On the other hand, the SS-DBSCAN clusters the marbles more deliberately by incorporating *Is_important*(*point*) condition. SS-DBSCAN produced 2 clusters with only 3 core points, as shown in Fig. 1b.

Finally, it's important to note that the time complexity of the SS-DBSCAN algorithm is $O(n^2)$ in its worst-case scenario. Adding the *Is_important* condition does not affect the time complexity, making it similar to the original DBSCAN.

IV. THE DEVELOPED CLUSTERING SYSTEM MODEL

The entire system model, which represents the experimental architecture, consists of three phases (preprocessing phase, clustering phase, and performance evaluation phase) as depicted in Fig. 2. The following subsections demonstrate a detailed explanation of each phase.

A. PREPROCESSING PHASE

Data preprocessing is applied to a dataset to ensure and enhance the clustering's performance. The preprocessing phase typically includes data cleaning, reducing dimensionality, feature transformation and engineering, data sampling, and handling imbalanced data.

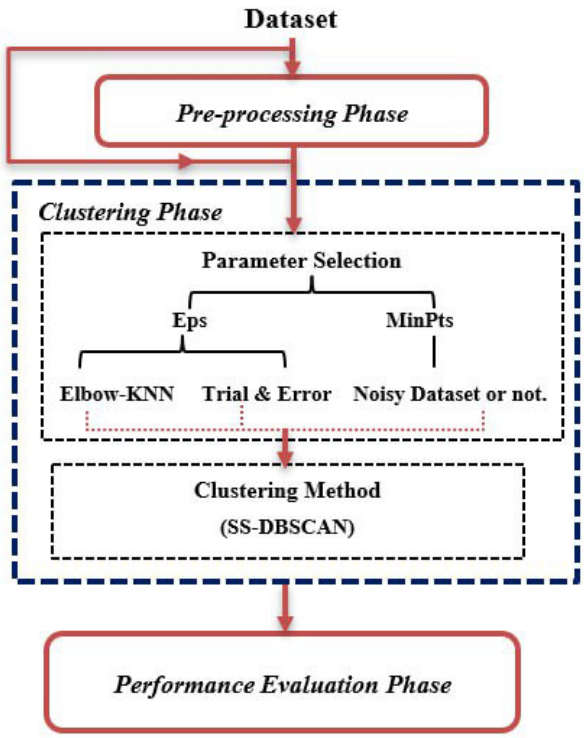


FIGURE 2. Main phases of the developed system model.

As mentioned previously, three real datasets are used to study the performance of SS-DBSCAN. Two UCI real datasets (Letter Recognition [15], and wireless localization [14]) are investigated with no preprocessing steps, as tested by [13] for concise comparison purposes. On the other hand, five preprocessing steps are applied to the raw Iraqi-MSA dataset: Cleaning, text normalization, tokenization, removing stop words, and stemming, as illustrated in [16].

B. CLUSTERING PHASE

The density-based algorithms require the specification of two parameters (*MinPts* and *Eps*) to accomplish the clustering process. Therefore, the clustering phase mainly consists of two stages:

Stage 1: Parameters Selection

At this stage, the parameters *MinPts* and *Eps* are determined. The *MinPts* parameter is selected based on whether

the data is noisy or not, as formulated in eq.(4) [43], where D is the number of features.

$$\text{MinPts} = \begin{cases} 2 \times D & : \text{Noisy data} \\ D + 1 & : \text{Noiseless data} \end{cases} \tag{4}$$

To select the best *Eps* parameter, two methods (Elbow-KNN) [44], [45] and trial and error are applied. Trial and error is used to verify whether Elbow-KNN can determine the appropriate *Eps* value. Epsilon *Eps* selection involves plotting the distances to the $k = MinPts$-1 nearest neighbor in descending order to create a k-distance graph. An effective *Eps* value is identified at the elbow of the plot. Choosing a very small *Eps* results in a significant portion of the data remaining unclustered, while a very large *Eps* causes clusters to blend, placing most points in a singular cluster. Preferably, *Eps* should be small, ensuring that only a limited number of points are within this distance from one another [45].

Stage 2: Apply the clustering algorithm (SS-DBSCAN) on the three datasets to study its performance.

C. PERFORMANCE EVALUATION PHASE

In this phase, we assess the clustering performance using either V-measure or perplexity (pp). The V-measure is used to assess the performance of a pre-classified dataset, thus it is suitable to measure the clustering performance of both letter recognition and wireless localization datasets. The SS-DBSCAN performance is compared to KR-DBSCAN, RNN-DBSCAN, ISBDBSCAN, ISDBSCAN, H-DBSCAN, and RECORD, in addition to the original DBSCAN algorithm in terms of V-measure. Furthermore, Perplexity (PP) is utilized to evaluate the clustering performance of the Iraqi-MSA dataset. PP serves as a language model performance metric and can be employed as an extrinsic clustering measure for non-pre-classified word datasets. A comparison of the performance achieved for the SS-DBSCAN, DBSCAN, RNN-DBSCAN, and H-DBSCAN is presented for the Iraqi-MSA dataset.

After computing the V-measure and the pp measure, two additional popular clustering metrics, the ARI, and the Silhouette score, are utilized to further evaluate SS-DBSCAN's effectiveness by comparing to the original DBSCAN, RNN-DBSCAN, and H-DBSCAN across the three datasets. A detailed description of the used measures is given below:

1) V-MEASURE

The V-measure is a metric derived from entropy, which evaluates the degree to which the homogeneity and completeness criteria are met as in eq. (5). To satisfy homogeneity, a clustering solution must place only data points from a single class together. Similarly, to satisfy completeness, a solution must ensure that all the data points belonging to the same class are placed in one cluster [46].

$$V = 2 \times \frac{h \times c}{h + c} \tag{5}$$

where h represents the homogeneity score as illustrated in eq. (6), and c is the completeness score as calculated in eq. (7).

$$h = 1 - \frac{\text{H}(C|K)}{\text{H}(C)} \tag{6}$$

$$c = 1 - \frac{\text{H}(K|C)}{\text{H}(K)} \tag{7}$$

where $\text{H}(C)$ is the entropy of the classes, and $\text{H}(K)$ is the entropy of the clusters.

The V-measure score is between 0.0 and 1.0. 1.0 stands for perfectly complete labeling.

2) THE ADJUSTED RAND INDEX (ARI)

ARI [47] compares the given classes with the generated clusters. A higher value indicates better agreement between the ground truth and the generated clusters. It measures the accuracy of determining whether a point belongs to a cluster, as in eq. (9). Rand Index measures the similarity between two clusterings by considering all pairs of points. The ARI compares the predicted and the ground truth clustering by counting the pairs of points assigned to clusters similar (or different) to how they are assigned in the true clusters.

Given a dataset sample of size n, $S = \{ o_1,\ldots,o_n \}$ where, o_i is one point in the dataset. Let $X = \{X_1,\ldots,X_r\}$ represents the ground truth clusters, where r is the number of the true clusters, and Let $Y = \{Y_1,\ldots,Y_m\}$ represents the predicted clusters, where m is the number of the predicted clusters. Consider the following definitions:

- a is the number of pairs of points that are in the same cluster in X and in the same cluster in Y too.
- b is the number of pairs that are in different clusters in X, and in different clusters in Y too.
- c is the number of pairs that are in the same cluster in X, but in different clusters in Y.
- d is the number of pairs that are in different clusters in X, but in the same cluster in Y.

The raw RI score calculated in eq. (8) is adjusted into the ARI score using the eq. (9):

$$\text{RI} = \frac{a + b}{a + b + c + d} \tag{8}$$

$$\text{ARI} = \frac{\text{RI-ExpectedRI}}{\text{Maximum(RI)-ExpectedRI}} \tag{9}$$

A 0 value of ARI indicates the worst agreement case between X and Y clusters, while a value of 1 indicates a full agreement between X and Y.

3) SILHOUETTE SCORE

The Silhouette score [48] does not require previously classified data to compare the clusters with the original classes. This metric reflects the compactness, separation, and variance of the clustering as shown in eq. (10):

$$\text{Sil} = \frac{b - a}{\text{Maximum}(b, a)} \tag{10}$$

where a is the mean distances inside one cluster, while b refers to the mean distances across the different clusters. A value near 1 indicates better clustering. On the other hand, the -1 value represents the worst case, and 0 indicates that the clusters are overlapping [47].

4) PERPLEXITY (PP)

The pp measure can be defined as the uncertainty in a probability distribution which represents a quantification of the cross entropy [49], (as formulated in eq. (12) and eq. (11)). A smaller pp means greater language model efficiency.

$$\mathrm{pp} = 2^{\mathrm{H}(P_{\mathrm{LM}})} \tag{11}$$

$$\mathrm{H}(P_{\mathrm{LM}}) = -\frac{1}{n}\sum_{i=1}^{n}\log(P_{\mathrm{LM}}(w_i|w_1\ldots w_{i-1})) \tag{12}$$

where for a given language model (LM), $P_{LM}\ (w_i|w_1\ldots w_{i-1})$ is the language model probability for a word w_i given that it was preceded with w_1 to w_{i-1} words sequence, and n is the total number of words.

V. EXPERIMENTAL SETUP

In this section, three case studies (datasets) are selected to examine the behaviour of SS-DBSCAN when applied to various kinds of data. Each case study has a unique core point condition and is presented with different characteristics to provide a comprehensive evaluation. The letters recognition dataset case is given to show the clustering behavior of an image data type. The wireless localization dataset case is utilized to test the clustering of the noisy signals. Lastly, the Arabic word dataset evaluates the clustering performance on a noiseless dataset, highlighting its effectiveness in language modeling scenarios. The description of each case study is demonstrated with the selection of the *MinPts*, and the specification of *Is_important* condition.

A. CASE STUDY 1: LETTERS RECOGNITION DATASET

Letters recognition from UCI datasets [15] is chosen as the first case study. The dataset contains 16 features based on pixels' statistics and is classified as one of the 26 English letters. The features represent statistical moments and edge counts scaled to fit a range of 0-15. The total size of instances is 20k letters. Table 1 shows four instances with a maximum value of each of the 16 features F1, F2,.., and F16.

TABLE 1. Four sample sets from the letters recognition dataset with their max statistics.

Features	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13	F14	F15	F16
Max	15	15	15	15	15	15	15	15	15	15	15	15	15	15	15	15
1	2	8	3	5	1	8	13	0	6	6	10	8	0	8	0	8
2	5	12	3	7	2	10	5	5	4	13	3	9	2	8	4	10
3	4	11	6	8	6	10	6	2	6	10	3	7	3	7	3	9
4	7	11	6	6	3	5	9	4	6	4	4	10	6	10	2	8

To check if a dataset is noisy, a graphical technique named the box plot method (the box-and-whisker plot) is used. The plot displays the data distribution through a box and whiskers design. Any point outside the whiskers is considered an outlier and represented as an individual point. The outliers existence affects the choice of the *MinPts* neighbors parameter as formulated in eq. (4).

Fig. 3 shows this dataset's number of outliers, which is relatively low compared to the large dataset size. Thus, we consider it a noiseless dataset. The *MinPts* parameter is set to D+1 totalling 17 based on eq. (4).

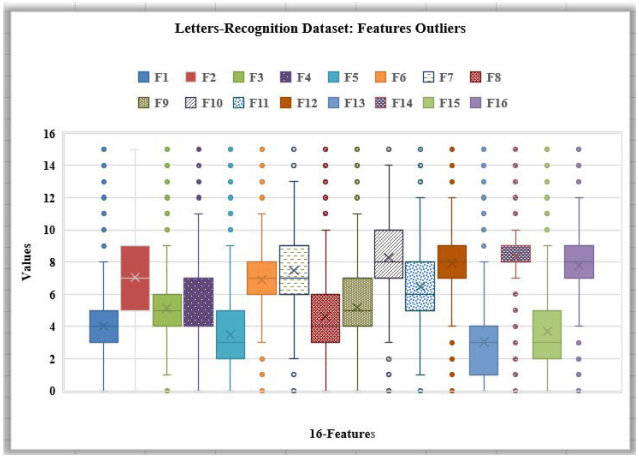


FIGURE 3. Letters recognition dataset's outliers.

The *Is_important* condition is specified based on the steps illustrated in section III:

- Step 1: The clustering aims to recognize a letter from its shape.
- Step 2: The criterion that makes the letter's image important is the degree of its clarity.
- Step 3: We utilized a feature selection algorithm, a trial-and-error approach, to select the key features since we could not directly specify them. This condition means that an instance letter can be a core if its image is clear, such that the pixels show contrast and their number is high. The point is considered important if one or more features has a value that is no less than a $maximum - 2$. As a result, features 1,2,3,4, and 10 were excluded from the *Is_important* function.
- Step 4: The proposed *Is_important* condition is defined in predicate (13).

$$\begin{aligned}&\forall x\ \mathrm{featureColumn}(x)\exists f\ \mathrm{feature}(f, l) \wedge (f \in x)\\&\quad\wedge (f \geq (\max(x) - 2))\\&\quad\Longrightarrow \mathrm{Is_important}(\mathrm{letter}(l))\end{aligned} \tag{13}$$

where l is one instance of the letters recognition dataset, x is the list of all instances of a feature (one column), and f is one feature that contributes to deciding whether l is a core point. The $feature(f, l)$ denotes any feature in the feature list. When any f is no smaller than the maximum feature's value -2, then l is important and can be considered a core point if the *MinPts* condition also applies.

B. CASE STUDY 2: WIRELESS INDOOR LOCALIZATION DATASET

The second case study is based on a wireless localization dataset used to specify the local location of a smartphone based on its WiFi signal strength [13], [14], [50]. The dataset size is 2k instances with seven features indicating the signal strengths. Table 2 lists four signal instances with x1, x2,.., and x7 features, each with its minimum and maximum signal strength.

TABLE 2. Four sample sets from the wireless localization dataset with their min and max statistics.

Features	x1	x2	x3	x4	x5	x6	x7
Min	-74	-74	-73	-77	-89	-97	-98
Max	-10	-45	-40	-11	-36	-61	-63
1	-64	-56	-61	-66	-71	-82	-81
2	-68	-57	-61	-65	-71	-85	-85
3	-63	-60	-60	-67	-76	-85	-84
4	-61	-60	-68	-62	-77	-90	-80

Since many points are outside the whiskers (i.e., outliers), as illustrated in Fig 4, the *MinPts* is set to 14 (2D) in light of eq. (4).

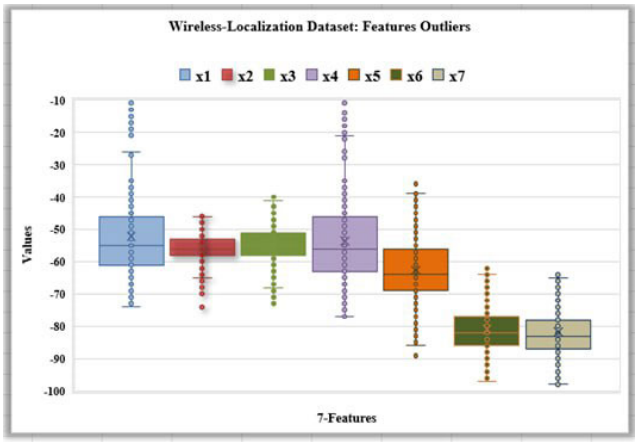


FIGURE 4. Wireless localization dataset's outliers.

To set the *Is_important* condition, the steps listed in section III are applied as follows:

- Step 1: The purpose of clustering is to specify the location of a smartphone based on its WiFi signal strength
- Step 2: The signal strength is the criterion, where strong signals indicate a nearby smartphone, while weak signals indicate a far located smartphone.
- Step 3: All features are considered since they all reflect a smartphone's signals that indicate its location (same importance). The smartphone is assumed to be important if one or more of the seven WiFi signals' strength is either extremely strong or extremely weak. Based on the WiFi data analysis, the extremely strong signal equals the strongest or, at most, two less than it. In contrast, the extremely weak signal is less than or equal to the weakest value plus two. The maximum and minimum values for each of the seven features are computed.
- Step 4: The proposed additional core point condition, *Is_important* condition, is as in predicate (14).

$$\begin{aligned}&\forall x\ \text{featureColumn}(x)\exists s\ \text{signal}(s,p)\wedge(s\in x)\\&\quad\wedge(s\geq(\max(x)-2)\vee(s\leq(\min(x)+2)))\\&\quad\Longrightarrow\ \text{Is_important}(\text{phone}(p))\end{aligned}\tag{14}$$

where $signal(s, p)$ denotes a WiFi signal of one instance (phone), and $featureColumn(x)$ is one of the seven features (i.e., a whole feature column).

C. CASE STUDY 3: ARABIC WORD IRAQI-MSA DATASET

This case study aims to cluster the small dialect words within the rich MSA relative words to create a class-based Language Model (LM). The purpose of this clustering is to transfer the information of the MSA language to its limited resources dialect, making the dialect dataset ready for further Natural Language Processing (NLP) applications.

The dataset is a combination of 1516k word MSA of the GALE recordings transcription taken from the Linguistic Data Consortium (LDC) [51], [52], and 12k word from the Iraqi Arabic conversational telephone speech transcription (LDC2006T16) [53] represented by Wasf-Vec feature vector. The Wasf-Vec embedding technique is a topology-based embedding [54]. The topology-based word embedding well fits the high morphology Arabic language. This is due to the variety of the same root word patterns and a reduced frequency of fixed-shape words that may appear in the dataset. This high morphology feature of the Arabic language causes the data sparsity issue [16]. For instance, a book reviewing document may include the words *book* kitAb, *his book* ktAbh, *written* mktwb, *author* kAtb, and *write* ktb. All these words are of the same root, and because of their similarity in their topology features, they will have feature vectors of near distances when using the Wasf-Vec word embedding. The same applies when embedding the dialect words that originated from the MSA. For example, the dialect word *give* AnTY will be near to the MSA word *give* AETY. Eventually, similar words should be clustered in the same cluster.

A dataset sample is illustrated in Table 3. Words are classified using the clustering algorithm. Each cluster number is considered a class. Classes are fed to a class-based language model. The language model pp is computed and considered as an efficiency metric.

TABLE 3. Four samples of arabic words dataset.

	word	Wasf vector[1]	Wasf vector[2]
1	trHyn	2808	10477
2	txAfyn	2676	10715
3	drwH	5086	4918
4	yrwH	12409	4922

This dataset is noiseless with no outliers outside the whiskers, as shown in Fig 5, thus *MinPts* is set to 3 (D+1) as in eq. (4).

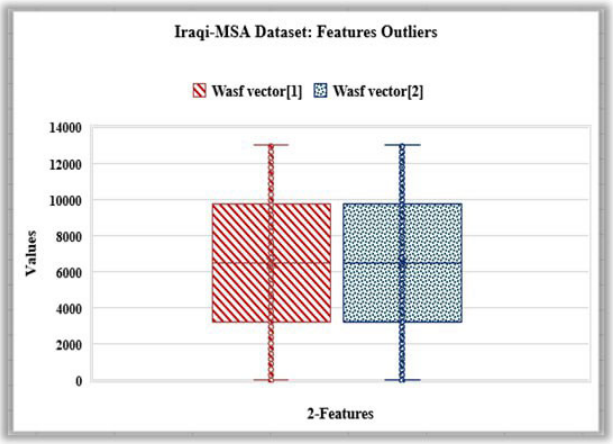


FIGURE 5. Arabic word Iraqi-MSA dataset's outliers.

The following steps summarize how the *Is_important* condition is formulated:

- Step 1: Use the clusters to produce a class-based language modeling.
- Step 2: The criterion defines a word as important if it appears in more than one common pattern. While the Arabic language is highly morphological, the word's clusters are similarly dense. This article proposes considering additional constraints for a point, which is a word, to be defined as a core point. The *Is_important* condition checks whether two or more different patterns of the same root words appear in the dataset.
- Step 3: The word letter sequence is the selected feature (word shape). The patterns selected in this research are fAEl and mfEwl, which are presented in Buckwalter transliteration [55]. For example, if the word *written* mktwb and the word *author* kAtb both exist in the dataset, then they should have special importance and are nominated to be core points in addition to have *MinPts* neighbors.
- Step 4: The predicate logic (15) shows the applied *Is_important* condition:

$$\begin{aligned}&\forall x\ \mathrm{word}(x) \wedge \mathrm{root}(x, r) \exists y\ \mathrm{word}(y)\\&\quad \wedge ((\mathrm{pattern}(x, r, \mathrm{fAEl}) \wedge \mathrm{pattern}(y, r, \mathrm{mFEwl}))\\&\quad \vee (\mathrm{pattern}(x, r, \mathrm{mFEwl}) \wedge \mathrm{pattern}(y, r, \mathrm{fAEl})))\\&\quad \Longrightarrow \mathrm{Is_important}(x)\end{aligned} \tag{15}$$

where x and y are words with root r, and the pattern of x is fAEl, and the pattern of y is mFEwl, or in the opposite way, where the pattern of y is fAEl, and the pattern of x is mFEwl. If these conditions are applied, then the word x is important.

VI. EXPERIMENTAL RESULTS

In this section, the effectiveness of SS-DBSCAN clustering of the three case studies is assessed. In subsection VI-A, we will compare the performance of SS-DBSCAN with the original DBSCAN, using V-measure or pp to determine *Eps*. The *MinPts* for each case study are specified using eq. (4), as described in section IV.

In subsection VI-B, we will compare SS-DBSCAN performance with six versions of DBSCAN in [13] in terms of V-measure. Additionally, a comparison will be made with the implemented open-source algorithms (DBSCAN, H-DBSCAN,[1] RNN-DBSCAN[2]) based on pp, ARI, and Silhouette score.

A. SS-DBSCAN VS. DBSCAN CLUSTERING PERFORMANCE

1) CASE STUDY 1: LETTERS RECOGNITION DATASET

Both the "trial and error" and the "Elbow value (Knee)" of the K-NN distance graph approaches are used to specify the *Eps* value. The two techniques agreed upon the same *Eps* value, which is 4 as shown in Fig. 6a. Fig. 6b presents different V-measure values across different *Eps* values starting from 3.8. The highest V-measure achieved by SS-DBSCAN is 0.533 when *Eps* equals 4. While the highest V-measure of the clusters generated by DBSCAN is 0.193 when *Eps* is 3.8. For both algorithms, the *MinPts* was set to 17.

The big improvement in the clustering performance of SS-DBSCAN over DBSCAN indicates that SS-DBSCAN succeeded in generating more accurate clusters by adding the *Is_important* condition. This condition reduced the number of core points, thus generating more precise clusters.

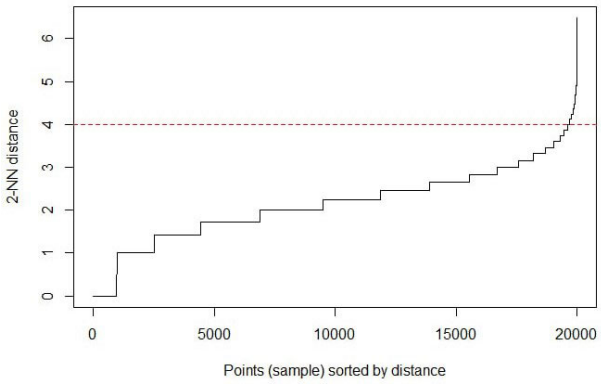
2) CASE STUDY 2: WIRELESS INDOOR LOCALIZATION DATASET

Using the Elbow method, the suggested best *Eps* is 15 based on the graph illustrated in Fig. 7a. However, this *Eps* value was tested, and it was found by trial and error that this is not the best choice of *Eps*. In this case, the Elbow value (Knee) of the K-NN distance is not useful for specifying the *Eps* value that improves clustering since sometimes there is more than one elbow or an unclear elbow in the graph. Thus, the *Eps* value specification is accomplished by trial and error. Fig. 7b shows that original DBSCAN and SS-DBSCAN clusters have similar V-measures. The best clustering (V-measure = 0.661) is achieved when *Eps* is set to 6.5 according to the trial and error technique, and *MinPts* is set to 14.

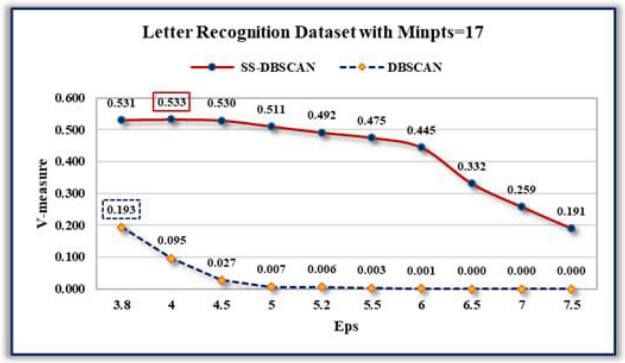
The original DBSCAN handles the exclusion of outliers (noisy data) through the allowance of a point to be a core when it has at least *MinPts* that are within a distance of *Eps*. Core points and their neighbors are clustered, while outliers are not assigned to any cluster. Thus, the DBSCAN and SS-DBSCAN demonstrated similar performance in handling

[1] https://pypi.org/project/hdbscan/

[2] https://github.com/tarigopulav/DataScience_Project3/blob/master/rnn_dbscan_Notes.py

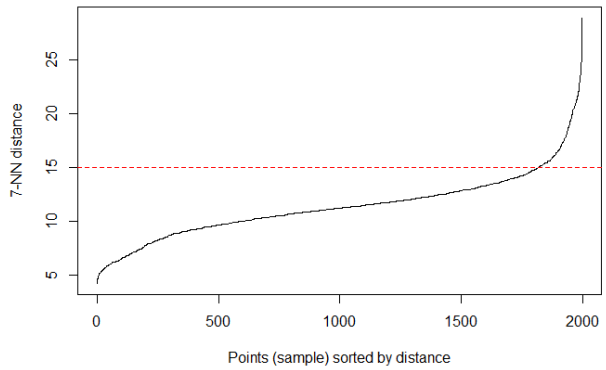


(a) Elbow K-NN distance graph implementation to decide *Eps* value

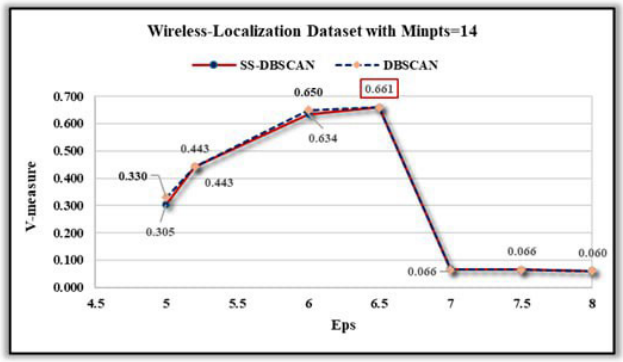


(b) V-measure

FIGURE 6. **Letters recognition dataset: Eps selection and results.**



(a) Elbow K-NN distance graph implementation to decide *Eps* value



(b) V-measure

FIGURE 7. **Wireless localization dataset: Eps selection and results.**

the noisy wireless localization dataset. However, including the *Is_important* condition did not enhance the clustering quality since the dataset was not large enough to generate more precise clusters.

3) CASE STUDY 3: ARABIC WORD IRAQI-MSA DATASET

In word clustering, an extrinsic evaluation was considered for computing clustering effectiveness [56]. The extrinsic evaluation is computed using the resulting clusters in another application and computing the application's performance. In this case study, the clustering performance is measured by a class-based language model's perplexity (pp). The related words (on the same cluster) share the statistical information in a class-based language model. This experiment mimics the work in [16] so that results can be compared with the original experiment. Having lower pp means better model efficiency.

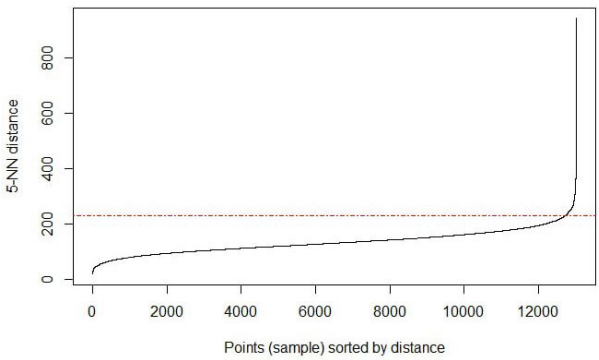
Both the "trial and error" and the "Elbow value (Knee) of the K-NN distance" agreed upon the same *Eps* value (250), and *MinPts* is set to 3. Using *Eps*=250 improves the clustering, as illustrated in Fig. 8b, where pp = 216.316.

Fig. 8 shows that the LM has improved by 14.5% using SS-DBSCAN. This reduction in pp indicates that the quality of the clusters has been increased by collecting more related words.

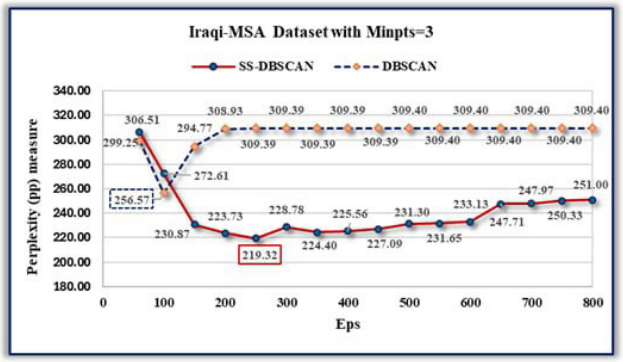
A core point would not be correctly identified based only on the vectors' distances due to the diversity of word patterns (sparsity issue). When applying the original DBSCAN, too many words are considered core points, resulting in solely one cluster for the whole dataset. On the other hand, when careful attention is given to the essential words, the clustering quality becomes higher, resulting in reduced language model pp.

B. SS-DBSCAN VS DBSCAN VERSIONS CLUSTERING PERFORMANCE

For further assessment of the effectiveness of SS-DBSCAN, we compared SS-DBSCAN and other versions of DBSCAN using metrics such as V-measure, pp, ARI, and Silhouette score. Initially, we compared the V-measure results of SS-DBSCAN with those published in [13]. The comparison included six versions of DBSCAN: KRDBSCAN, RNN-DBSCAN, ISBDBSCAN, RECORD, H-DBSCAN, in addition to the original DBSCAN and SS-DBSCAN, as illustrated in Fig 9. For the letters recognition dataset, SS-DBSCAN shows comparable performance to the highest V-measure achieved in [13]. Similarly, for the wireless localization dataset, the V-measure achieved by SS-DBSCAN is comparable to that achieved by DBSCAN and RNN-DBSCAN. The V-measure values show that the clusters generated



(a) Elbow K-NN distance graph implementation to decide *Eps* value



(b) Perplexity (pp) measure

FIGURE 8. Arabic word Iraqi-MSA dataset: Eps selection and results.

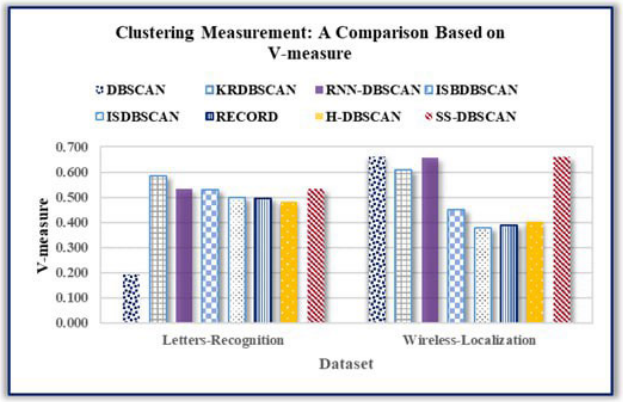


FIGURE 9. V-measure comparison of letters recognition and wireless localization datasets clustering using versions of DBSCAN.

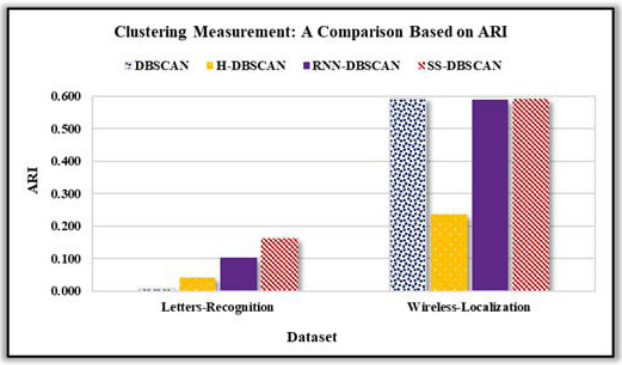


FIGURE 10. ARI scores for letters recognition and wireless localization datasets clusters by applying DBSCAN, RNN-DBSCAN, H-DBSCAN, and SS-DBSCAN.

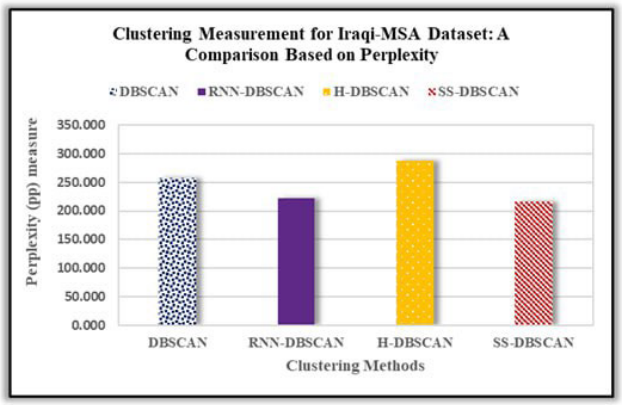


FIGURE 11. Perplexity (pp) measure of DBSCAN, RNN-DBSCAN, H-DBSCAN, and SS-DBSCAN (Lower is better).

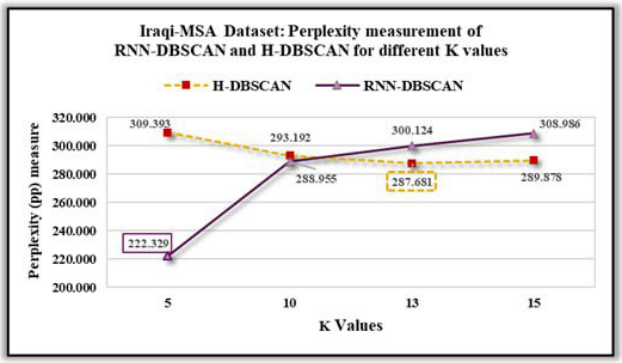


FIGURE 12. LM pp when clustering using RNN-DBSCAN and H-DBSCAN for various values of *K*.

by SS-DBSCAN outperform those from other DBSCAN versions.

Furthermore, we compare DBSCAN and SS-DBSCAN, as well as two open-source versions of DBSCAN (RNN-DBSCAN and H-DBSCAN), based on the ARI measures using letters recognition and wireless localization datasets also. The results, shown in Fig. 10, display the ARI scores for four versions of DBSCAN. The SS-DBSCAN algorithm performed better than the other three versions. The ARI scores for DB-SCAN, RNN-DBSCAN, and SS-DBSCAN were almost equal, while the ARI score for H-DBSCAN was lower. Since no previous classification was given for the Iraqi-MSA dataset, no ARI was computed.

For the extrinsic LM evaluation (pp), the comparison chart shown in Fig. 11 indicates that SS-DBSCAN performs better than the other three versions. The clusters produced by RNN-DBSCAN with a K value of 5 had the lowest LM

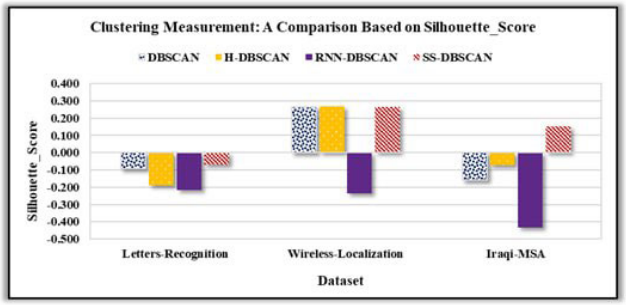


FIGURE 13. Silhouette scores for all datasets clusters by applying DBSCAN, RNN-DBSCAN, H-DBSCAN, and SS-DBSCAN.

pp of 222.3285, while H-DBSCAN clusters resulted in pp of 287.6813 when k = 13, as illustrated in Fig. 12. The reason behind generating Fig. 12 is to reach the best k values for H-DBSCAN and RNN-DBSCAN algorithms that give the highest pp. In order to compare SS-DBSCAN with the best pp measures returned by these two algorithms.

Moreover, the Silhouette score is calculated for the four versions of DBSCAN after applying them to the three datasets, as shown in Fig. 13. The RNN-DBSCAN has the lowest Silhouette score across the three datasets. However, the SS-DBSCAN performs the best when applied to the Letter-Recognition and Iraqi-MSA datasets. For the wireless localization dataset, DBSCA, SS-DBSCAN, and H-DBSCAN have nearly equal Silhouette scores.

The substantial enhancement in clustering performance suggests that SS-DBSCAN successfully generated more accurate clusters by incorporating the *Is_important* condition.

VII. CONCLUSION

A novel variant of the well-established DBSCAN clustering algorithm is introduced. The SS-DBSCAN algorithm greatly improves the accuracy of cluster identification by selecting core points based on specific criteria through adding *Is_important* constraint. To evaluate the effectiveness of the suggested modification, SS-DBSCAN is applied to three different types of datasets: The letters recognition dataset (relatively noiseless), the wireless localization dataset (noisy), and the Iraqi-MSA dataset to produce a class-based LM (noiseless).

This constraint allows the incorporation of human insight to align with the unique objectives and attributes of the dataset, resulting in a customized and objective-based (more refined and adaptable) clustering approach. This predefined condition is set differently depending on the criteria of the three underlying case studies. For each of the three case studies, the *Is_important* conditions are set based on prioritizing features. In all cases, we have designed the *Is_important* condition such that it can identify a core point as a point having extreme feature value. Thus, for any other dataset, it must be decided what makes a point special and suitable to represent a cluster. This can identify more meaningful clusters. Statistical analysis of a dataset can help identify special points, and different values can be considered based on the purpose of clustering.

Similar to DBSCAN, the SS-DBSCAN algorithm requires two parameters to be specified, *MinPts* and *Eps*. The first parameter is selected based on the noisiness status of the dataset (noisy or noiseless). The second parameter can be determined by using trial and error as well as the Elbow value (Knee) of the K-NN distance graphs technique. This can help in deciding whether the Elbow value (Knee) is adequate for determining the best *Eps* value or not. The results show that the K-NN distance graph's Elbow value (Knee) does not always help determine the best *Eps* value.

SS-DBSCAN greatly enhanced the clustering V-measure of the letters recognition dataset by 65%. On the other hand, testing it on the noisy wireless localization dataset reached comparative results to the original algorithm, DBSCAN. The clustering analysis of the Iraqi-MSA dataset is measured using the perplexity of the language model based on the produced clusters. Experimental results also showed that SS-DBSCAN is significantly effective in clustering these noiseless datasets, achieving a 14.5% improvement over the original DBSCAN algorithm. Moreover, the SS-DBSCAN is compared with other versions of DBSCAN. The experimental results demonstrate that SS-DBSCAN outperforms most DBSCAN versions in the three case studies based on four metrics: V-measure, pp, ARI, and Silhouette score.

For future work, it is recommended to test the SS-DBSCAN using other *Is_important* condition setups. In Addition, the *Is_important* condition can be merged with other versions of DBSCAN to improve their performance. Moreover, optimizing memory usage plays a crucial role in further work on SS-DBSCAN because the existing approach is rather memory-intensive and ineffective within big datasets. Thus, it can be implemented using distributed computing technology.

REFERENCES

[1] N. S. Sagheer and S. A. Yousif, "Canopy with k-means clustering algorithm for big data analytics," in *Proc. AIP Conf.*, vol. 2334, 2021, p. 70006.

[2] E. Güngör and A. Özmen, "Distance and density based clustering algorithm using Gaussian kernel," *Exp. Syst. Appl.*, vol. 69, pp. 10–20, Mar. 2017.

[3] C. Zhang, W. Huang, T. Niu, Z. Liu, G. Li, and D. Cao, "Review of clustering technology and its application in coordinating vehicle subsystems," *Automot. Innov.*, vol. 6, pp. 1–27, Jan. 2023.

[4] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, vol. 1, Oakland, CA, USA, 1967, pp. 281–297.

[5] K. Khan, S. U. Rehman, K. Aziz, S. Fong, and S. Sarasvady, "DBSCAN: Past, present and future," in *Proc. 5th Int. Conf. Appl. Digit. Inf. Web Technol. (ICADIWT)*, Feb. 2014, pp. 232–238.

[6] G. Karypis, E. Han, and V. Kumar, "A hierarchical clustering algorithm using dynamic modeling," Univ. Digit. Conservancy, 1999. [Online]. Available: https://hdl.handle.net/11299/215363

[7] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. KDD*, vol. 96, 1996, pp. 226–231.

[8] D. Fitrianah, H. Fahmi, A. N. Hidayanto, and A. M. Arymurthy, "Improved partitioning technique for density cube-based spatio-temporal clustering method," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 10, pp. 8234–8244, Nov. 2022.

[9] S. Li, L. Li, J. Yan, and H. He, "SDE: A novel clustering framework based on sparsity-density entropy," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 8, pp. 1575–1587, Aug. 2018.

[10] M. Gogebakan and H. Erol, "A new semi-supervised classification method based on mixture model clustering for classification of multispectral data," *J. Indian Soc. Remote Sens.*, vol. 46, no. 8, pp. 1323–1331, Aug. 2018.

[11] M. Gogebakan, "A novel approach for Gaussian mixture model clustering based on soft computing method," *IEEE Access*, vol. 9, pp. 159987–160003, 2021.

[12] H. A. Chowdhury, D. K. Bhattacharyya, and J. K. Kalita, "UIFDBC: Effective density based clustering to find clusters of arbitrary shapes without user input," *Exp. Syst. Appl.*, vol. 186, Dec. 2021, Art. no. 115746.

[13] L. Hu, H. Liu, J. Zhang, and A. Liu, "KR-DBSCAN: A density-based clustering algorithm based on reverse nearest neighbor and influence space," *Exp. Syst. Appl.*, vol. 186, Dec. 2021, Art. no. 115763.

[14] D. Dua and C. Graff, "UCI machine learning repository," School Inf. Comput. Sci., Univ. California, Irvine, 2017. [Online]. Available: http://archive.ics.uci.edu/ml

[15] P. W. Frey and D. J. Slate, "Letter recognition using holland-style adaptive classifiers," *Mach. Learn.*, vol. 6, no. 2, pp. 161–182, Mar. 1991.

[16] T. Z. Abdulhameed, "Cross language information transfer between modern standard Arabic and its dialects—A framework for automatic speech recognition system language model," Ph.D. thesis, Western Michigan Univ., MI, USA, 2020.

[17] H. V. Singh, A. Girdhar, and S. Dahiya, "A literature survey based on DBSCAN algorithms," in *Proc. 6th Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, May 2022, pp. 751–758.

[18] X. Yu, D. Zhou, and Y. Zhou, "A new clustering algorithm based on distance and density," in *Proc. Int. Conf. Services Syst. Services Manage.*, vol. 2, 2005, pp. 1016–1021.

[19] P. Liu, D. Zhou, and N. Wu, "VDBSCAN: Varied density based spatial clustering of applications with noise," in *Proc. Int. Conf. Service Syst. Service Manage.*, Jun. 2007, pp. 1–4.

[20] O. Uncu, W. A. Gruver, D. B. Kotak, D. Sabaz, Z. Alibhai, and C. Ng, "GRIDBSCAN: GRId density-based spatial clustering of applications with noise," in *Proc. IEEE Int. Conf. Syst., Man Cybern.*, Oct. 2006, pp. 2976–2981.

[21] S. Mahran and K. Mahar, "Using grid for accelerating density-based clustering," in *Proc. 8th IEEE Int. Conf. Comput. Inf. Technol.*, Jul. 2008, pp. 35–40.

[22] X. Lu-Ning and J. Ji-Wu, "SA-DBSCAN: A self-adaptive density-based clustering algorithm," *J. Univ. Chin. Acad. Sci.*, vol. 26, no. 4, p. 530, 2009.

[23] B. Borah and D. K. Bhattacharyya, "An improved sampling-based DBSCAN for large spatial databases," in *Proc. Int. Conf. Intell. Sens. Inf. Process.*, 2004, pp. 92–96.

[24] D. Birant and A. Kut, "ST-DBSCAN: An algorithm for clustering spatial–temporal data," *Data Knowl. Eng.*, vol. 60, no. 1, pp. 208–221, Jan. 2007.

[25] V. Mistry, U. Pandya, A. Rathwa, H. Kachroo, and A. Jivani, "Aedbscan-adaptive epsilon density-based spatial clustering of applications with noise," in *Progress in Advanced Computing and Intelligent Engineering*, vol. 2. Berlin, Germany: Springer, 2021, pp. 213–226.

[26] K. Giri and T. K. Biswas, "Determining optimal epsilon (EPS) on DBSCAN using empty circles," in *Proc. Int. Conf. Artif. Intell. Sustain. Eng.*, vol. 1. Singapore: Springer, 2022, pp. 265–275.

[27] C. Xiaoyun, M. Yufang, Z. Yan, and W. Ping, "GMDBSCAN: Multi-density DBSCAN cluster based on grid," in *Proc. IEEE Int. Conf. e-Business Eng.*, Oct. 2008, pp. 780–783.

[28] S.-S. Li, "An improved DBSCAN algorithm based on the neighbor similarity and fast nearest neighbor query," *IEEE Access*, vol. 8, pp. 47468–47476, 2020.

[29] H. Yatish, M. P. Shubham, T. S. Hukkeri, L. Xu, G. Shobha, J. Shetty, and A. Chala, "Massively scalable density based clustering (DBSCAN) on the HPCC systems big data platform," *IAES Int. J. Artif. Intell. (IJ-AI)*, vol. 10, no. 1, p. 207, Mar. 2021.

[30] A. Ram, A. Sharma, A. S. Jalal, A. Agrawal, and R. Singh, "An enhanced density based spatial clustering of applications with noise," in *Proc. IEEE Int. Advance Comput. Conf.*, Mar. 2009, pp. 1475–1478.

[31] B. Borah and D. K. Bhattacharyya, "A clustering technique using density difference," in *Proc. Int. Conf. Signal Process., Commun. Netw.*, Feb. 2007, pp. 585–588.

[32] R. J. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. Pacific–Asia Conf. Knowl. Discovery Data Mining*. Berlin, Germany: Springer, 2013, pp. 160–172.

[33] S. Vadapalli, S. Valluri, and K. Karlapalem, "A simple yet effective data clustering algorithm," in *Proc. 6th Int. Conf. Data Mining (ICDM)*, Dec. 2006, pp. 1108–1112.

[34] A. Bryant and K. Cios, "RNN-DBSCAN: A density-based clustering algorithm using reverse nearest neighbor density estimates," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 6, pp. 1109–1121, Jun. 2018.

[35] C. Cassisi, A. Ferro, R. Giugno, G. Pigola, and A. Pulvirenti, "Enhancing density-based clustering: Parameter reduction and outlier detection," *Inf. Syst.*, vol. 38, no. 3, pp. 317–330, May 2013.

[36] Y. Lv, T. Ma, M. Tang, J. Cao, Y. Tian, A. Al-Dhelaan, and M. Al-Rodhaan, "An efficient and scalable density-based clustering algorithm for datasets with complex structures," *Neurocomputing*, vol. 171, pp. 9–22, Jan. 2016.

[37] C. Ruiz, M. Spiliopoulou, and E. Menasalvas, "Density-based semi-supervised clustering," *Data Mining Knowl. Discovery*, vol. 21, no. 3, pp. 345–370, Nov. 2010.

[38] J. Wang, C. Zhu, Y. Zhou, X. Zhu, Y. Wang, and W. Zhang, "From partition-based clustering to density-based clustering: Fast find clusters with diverse shapes and densities in spatial databases," *IEEE Access*, vol. 6, pp. 1718–1729, 2018.

[39] T. N. Tran, K. Drab, and M. Daszykowski, "Revised DBSCAN algorithm to cluster data with dense adjacent clusters," *Chemometric Intell. Lab. Syst.*, vol. 120, pp. 92–96, Jan. 2013.

[40] N. Ohadi, A. Kamandi, M. Shabankhah, S. M. Fatemi, S. M. Hosseini, and A. Mahmoudi, "SW-DBSCAN: A grid-based DBSCAN algorithm for large datasets," in *Proc. 6th Int. Conf. Web Res. (ICWR)*, Apr. 2020, pp. 139–145.

[41] Z. Cai, J. Wang, and K. He, "Adaptive density-based spatial clustering for massive data analysis," *IEEE Access*, vol. 8, pp. 23346–23358, 2020.

[42] A. Fahim, "An extended DBSCAN clustering algorithm," *Int. J. Adv. Comput. Sci. Appl.*, vol. 13, no. 3, pp. 245–258, 2022.

[43] C. Maklin, "DBSCAN Python example: The optimal value for epsilon (EPS)," *Towards Data Sci.*, pp. 245–258, 2019.

[44] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "DBSCAN revisited, revisited: Why and how you should (Still) use DBSCAN," *ACM Trans. Database Syst.*, vol. 42, no. 3, pp. 1–21, Sep. 2017.

[45] D. Toshniwal, N. Chaturvedi, M. Parida, A. Garg, C. Choudhary, and Y. Choudhary, "Application of clustering algorithms for spatio-temporal analysis of urban traffic data," *Transp. Res. Proc.*, vol. 48, pp. 1046–1059, Jan. 2020.

[46] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure," in *Proc. Joint Conf. Empir. Methods Natural Lang. Process. Comput. Natural Lang. Learn. (EMNLP-CoNLL)*, 2007, pp. 410–420.

[47] K. M. Kanaujia, A. Srigyan, U. Mishra, S. Sirvi, and S. J. Nanda, "Robust automatic clustering based on local density with glowworm swarm optimization," in *Proc. 12th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jul. 2021, pp. 1–7.

[48] K. R. Shahapure and C. Nicholas, "Cluster quality analysis using silhouette score," in *Proc. IEEE 7th Int. Conf. Data Sci. Adv. Analytics (DSAA)*, Oct. 2020, pp. 747–748.

[49] I. Heintz and I. Zitouni, "Language modeling," in *Natural Language Processing of Semitic Languages* (Theory and Applications of Natural Language Processing), I. Zitouni, Ed. Berlin, Germany: Springer, 2014, pp. 161–196.

[50] J. G. Rohra, B. Perumal, S. J. Narayanan, P. Thakur, and R. B. Bhatt, "User localization in an indoor environment using fuzzy hybrid of particle swarm optimization & gravitational search algorithm with neural networks," in *Proc. 6th Int. Conf. Soft Comput. Problem Solving* (Advances in Intelligent Systems and Computing), vol. 1, K. Deep et al., Singapore: Springer, 2017, doi: 10.1007/978-981-10-3322-3_27.

[51] M. Glenn, H. Lee, S. Strassel, and M. Kazuaki, "GALE phase 2 Arabic broadcast conversation transcripts—Part 1 LDC2013T04," Linguistic Data Consortium, Philadelphia, PA, USA, 2013. [Online]. Available: https://catalog.ldc.upenn.edu/LDC2013T04

[52] M. Glenn, H. Lee, S. Strassel, and K. Maeda, "GALE phase 2 Arabic broadcast conversation transcripts—Part 2 LDC2013T17," Linguistic Data Consortium, Philadelphia, PA, USA, 2013. [Online]. Available: https://catalog.ldc.upenn.edu/LDC2013T17

[53] A. Appen, "Iraqi Arabic conversational telephone speech, transcripts LDC2006T16," Linguistic Data Consortium, Philadelphia, PA, USA, Tech. Rep., 2006.

[54] T. Z. Abdulhameed, I. Zitouni, and I. Abdel-Qader, "Wasf-vec: Topology-based word embedding for modern standard Arabic and Iraqi dialect ontology," *ACM Trans. Asian Low-Resource Lang. Inf. Process.*, vol. 19, no. 2, pp. 1–27, Dec. 2019.

[55] N. Habash, A. Soudi, and T. Buckwalter, "On Arabic transliteration," *Arabic Computational Morphology: Knowledge-Based and Empirical Methods*. New York, NY, USA: Association for Computing Machinery, 2007, pp. 15–22.
[56] E. Amigó, J. Gonzalo, J. Artiles, and F. Verdejo, "A comparison of extrinsic clustering evaluation metrics based on formal constraints," *Inf. Retr.*, vol. 12, no. 5, p. 613, Oct. 2009.

TIBA ZAKI ABDULHAMEED (Member, IEEE) received the B.Sc. and M.Sc. degrees from the College of Sciences, Al-Nahrain University, and the Ph.D. degree from the College of Engineering and Applied Sciences, Western Michigan University, in 2020. She is currently a Lecturer with the College of Sciences, Al-Nahrain University. Her current interests include machine learning, natural language processing, language modeling, and automatic speech recognition systems.

SUHAD A. YOUSIF received the B.Sc. degree from Al-Nahrain University, in 1994, the M.Sc. degree from the Computer Science Department, Baghdad University, in 2005, and the Ph.D. degree from the Mathematics and Computer Science Department, Beirut Arab University, Lebanon, in 2015. She is currently an Assistant Professor with the College of Science, Al-Nahrain University. She supervises M.Sc. theses concerning cloud computing, big data analysis, text classification (natural language processing), classification of ensemble machine learning, automated machine learning, and forecasting prediction. She also leads and teaches different subjects at both B.Sc. and M.Sc. levels in computer science. Her research interests include big data science, machine learning, and deep learning. In addition, she is on a scientific committee at some conferences and has been a reviewer at several conferences and journals.

VENUS W. SAMAWI received the B.Sc. degree from the University of Technology, in 1987, and the M.Sc. and Ph.D. degrees from the Computer Science Department, Al-Nahrain University, in 1992 and 1999, respectively. She is currently a Full Professor with the Department of Management Information Systems/Smart Business (Master Program), Isra University. She became a member of the International Association of Engineers (IAENG). She supervises many M.Sc. and Ph.D. theses concerning system programming, pattern recognition, network security, and text classification. She also leads and teaches modules at the B.Sc. and M.Sc. levels in computer science. Her special research interests include pattern recognition, evolutionary computing, image processing, natural language processing, and the IoT. She is a reviewer of several conferences and journals.

HASNAA IMAD AL-SHAIKHLI received the B.Sc. and M.Sc. degrees from the Computer Science Department, College of Sciences, Al-Nahrain University, in 2005 and 2009, respectively, and the Ph.D. degree in bioinformatics from the Computer Science Department, College of Engineering and Applied Sciences, Western Michigan University, in 2019. She is currently a Lecturer with the Computer Science Department, Al-Nahrain University. Her research interests include bioinformatics, computational biology, algorithm design and analysis, machine learning, and deep learning.